

Relatório

Integrantes: Guilherme Casarino Maia, Guilherme Genicy Camargos, Pedro Paulo Gomes Martins.

Introdução

O tema escolhido para o nosso projeto prático foi times de futebol, nele implementamos os recursos aprendidos em sala de aula. Nosso objetivo com este trabalho é desenvolver um programa capaz de gerenciar as informações de um clube, sendo elas: a posição em um ranking de clubes, o nome do clube, onde o clube está localizado, o seu ano de fundação e quantidade de títulos adquiridos.

Estruturas Utilizadas

O programa utiliza um registro ([struct](#)) chamado TimeFutebol para armazenar as informações sobre cada um dos times. Os campos utilizados são:

- **identificador:** um inteiro ([int](#)) representando a posição do time no ranking.
- **nome:** um [char](#) de no máximo 40 caracteres armazenando o nome do clube.
- **local:** um [char](#) de no máximo 40 caracteres informando a cidade e o país do clube.
- **anoFundacao:** um inteiro ([int](#)) que representa o ano de fundação do clube.
- **titulos:** um inteiro ([int](#)) que representa o total de títulos conquistados de cada clube.

Leitura e escrita dos arquivos

Primeiramente, o código lê todos os dados de um arquivo CSV denominado TimesFutebol.csv, onde estão armazenadas as informações de 100 clubes inicialmente. Após essa leitura, é possível armazenar esses dados em um arquivo binário, que será aberto para realizar as alterações necessárias.

Os procedimentos e funções utilizadas para realizar a leitura do arquivo CSV, a escrita em um arquivo binário e a leitura desse arquivo são respectivamente:

- [TimeFutebol*](#) leituraArquivo
- [void](#) salvarEmBinario
- [TimeFutebol*](#) leituraBinario

Busca Binária

Para realizar as buscas, optamos por utilizar os campos de posição no ranking e nome do clube. Implementamos uma busca binária iterativa, onde a variável meio é calculada somando-se a posição inicial com a posição final e dividindo o resultado por 2. Se o ranking procurado for maior que o valor na posição meio, a nova posição inicial será ajustada para (meio + 1), concentrando a busca na metade superior do conjunto de dados. Caso contrário, a posição final é ajustada para (meio - 1), restringindo a busca à metade inferior.

Procedimentos utilizados para fazer essa busca binária.

- `void` BuscaRanking
- `void` BuscaNomeClube

Ordenação

Para ordenar o arquivo, utilizamos o método eficiente de quick sort. Os campos escolhidos para a ordenação são: ano de fundação e títulos, ambos em ordem crescente e decrescente; nome do clube, em ordem alfabética crescente e decrescente; e posição no ranking (identificador), em ordem crescente.

Funções e procedimentos utilizados para fazer a ordenação dos títulos.

- `int` ParticionamentoTitulos_Crescente
- `void` QuickSortTitulosCrescente
- `int` ParticionamentoTitulos_Decrescente
- `void` QuickSortTitulosDecrescente

Funções e procedimentos utilizados para fazer a ordenação do nome de cada clube.

- `int` ParticionamentoClubes_Crescente
- `void` QuickSortClubesCrescente
- `int` ParticionamentoClubes_Decrescentes
- `void` QuickSortClubesDecrescente

Funções e procedimentos utilizados para fazer a ordenação do ano de fundação de cada um dos clubes.

- `int` ParticionamentoAno_Crescente
- `void` QuickSortAnoCrescente
- `int` ParticionamentoAno_Decrescente
- `void` QuickSortAnoDecrescente

Função e procedimento utilizado para fazer a ordenação do ranking em ordem crescente.

- `int` ParticionamentoPosicao_Crescente
- `void` QuickSortPosicaoCrescente

Inserção e Exclusão

Para inserir um novo clube, é necessário fornecer o nome, a localização do clube, o ano de fundação e o número de títulos. A posição no ranking será sempre atribuída à última posição do arquivo.

- `void` NovoElemento

A exclusão de um clube do arquivo é feita através da posição do clube, por exemplo: o Real Madrid CF ocupa a primeira posição no ranking, se a gente o excluir, todo o arquivo receberá uma alteração, pois a posição no ranking de todo clube que está acima dele irá diminuir 1.

- `void` ExcluirElemento

Outros procedimentos foram utilizados, sendo eles:

- `void` MostrarArquivo
- `void` PesquisarPosicao
- `void` GravarArquivo

MostrarArquivo serve para imprimir os dados armazenados no código.

PesquisarPosicao, é utilizado para pesquisar a partir de uma posição inicial até uma posição final. E GravarArquivo é um procedimento que é utilizado para gravar as alterações no arquivo.

Na função `main`, utilizamos dois blocos `if`, o primeiro cria um arquivo binário caso ele não exista, e o segundo lê o arquivo binário e executa um `switch`, que serve como a interface principal do programa.

Acertos e erros durante o desenvolvimento

Durante o trabalho, corrigimos dois erros principais: a falha na atualização dos rankings (identificador) ao excluir elementos e a inserção incorreta de dados no arquivo binário após ler o CSV. A boa organização do código facilitou identificar e resolver os problemas de forma mais eficiente.

Conclusão

Apesar dos desafios que enfrentamos, como atualizar os rankings e adicionar dados aos arquivos, conseguimos alcançar um bom resultado. O programa está funcionando como o esperado e gerenciando as informações dos clubes de futebol de forma eficiente, com todas as funcionalidades que planejamos.